

# Lab 01 R Learning

Siddhesh Shinde

r Sys.Date()

```
{r machine-info, echo=FALSE, results='asis'} cat("**Machine:**",
paste0(Sys.info()['nodename'], " - ", Sys.info()['sysname'], " ",
Sys.info()['release']), "\n\n")

{r setup, include=FALSE} knitr::opts_chunk$set(echo = TRUE) # Set
machine info for document machine_info <- paste0(Sys.info()['nodename'],
' - ', Sys.info()['sysname'], ' ', Sys.info()['release'])
```

## Logical Operators:

1. Use logical operations to get R to agree that “two plus two equals 5” is FALSE.

I have used “==” operator to compare the values “2+2” and “5” to show that they are not equal and it returns FALSE

```
{r q1} 2+2 == 5
```

2. Use logical operations to test whether  $8^{13}$  is less than  $15^9$ .

I have used “<” operator to compare the values “ $8^{13}$ ” and “ $15^9$ ” to show that “ $8^{13}$ ” is not less than “ $15^9$ ” the operation returns FALSE

```
{r q2} 8^13 < 15^9
```

## Variables:

3. Create a variable called potato whose value corresponds to the number of potatoes you’ve eaten in the last week. Or something equally ridiculous. Print out the value of potato.

I have used “<-” operator to assign the value 10000000000 to a variable called potato

```
{r q3} potato <- 10000000000 potato
```

4. Calculate the square root of potato using the `sqrt()` function. Print out the value of potato again to verify that the value of potato hasn't changed.

Using `sqrt(potato)` we get square root of potato i.e. `sqrt(10000000000)` which is "1e+05" The value of potato has not changed. It is still the original value: "1e+10"

```
{r q4} sqrt(potato) potato
```

5. Reassign the value of potato to `potato * 2`. Print out the new value of potato to verify that it has changed.

On reassigning the value of potato to `potato*2` using the "<-" operator, we get the new value of potato: "2e+10"

```
{r q5} potato <- potato * 2 potato
```

6. Try making a character (string) variable and a logical variable. Try creating a variable with a "missing" value NA. You can call these variables whatever you would like. Use `class(variablename)` to make sure they are the right type of variable.

Using "<-" operator to assign values, I have created character variable called "char", logical variable called "logical" and missing variable called "missing". Using `class()`, we find that "Hello World!" is a character, TRUE is logical and NA is logical as well.

```
“{r q6} char <- "Hello World!" class(char)
```

```
logical <- TRUE class(logical)
```

```
missing <- NA class(missing)
```

**# Vectors:**

7. Create a numeric vector with three elements using `c()`.

I used `c()` to create a numeric vector called "num\_vec" with values 1, 2 and 3

```
```{r q7}  
num_vec <- c(1, 2, 3)  
num_vec
```

8. Create a character vector with three elements using `c()`.

I used `c()` to create a character vector called "char\_vec" with values "a", "b" and "c"

```
{r q8} char_vec <- c("a", "b", "c") char_vec
```

9. Create a numeric vector called age whose elements contain the ages of three people you know, where the names of each element correspond to

the names of those people.

I created a numeric vector “age” and used “names()” to assign names to each element of age

```
{r q9} age <- c(19, 34, 41) names(age) <- c("Alice", "Bob",  
"Charlie") age
```

10. Use “indexing by number” to get R to print out the first element of one of the vectors you created in the last questions.

Using “[1]”, I indexed the first element of “age”

```
{r q10} age[1]
```

11. Use logical indexing to return all the ages of all people in age greater than 20.

Using “age > 20” returns TRUE for index where the age is greater than 20. I used this as a logical index for getting all ages greater than 20 from “age”

```
{r q11} age[age > 20]
```

12. Use indexing by name to return the age of one of the people whose ages you’ve stored in age

Using the name of the person as an index, I got age of Alice

```
{r q12} age["Alice"]
```

## Matrices:

## Dataframes:

13. Load the airquality dataset.
14. Use the \$ method to print out the Wind variable in airquality.
15. Print out the third element of the Wind variable.

I used the “data()” method to load the airquality dataset. Accessing “Wind” variable required using Wind after \$ from airquality which print the dataframe  
Accessing 3rd element can be achieved using “[3]” after Wind

```
{r q13-15} data(airquality) airquality$Wind airquality$Wind[3]
```

16. Create a new data frame called aq that includes only the first 10 cases.  
Hint: typing c(1,2,3,4,5,6,7,8,9,10) is tedious. R allows you to use 1:10 as a shorthand method!
17. Use logical indexing to print out all days (ie. cases) in aq where the Ozone level was higher than 20.
  - a. What did the output do with NA values?
18. Use subset() to do the same thing. Notice the difference in the output.

I used “[1:10, ]” to select rows from 1 to 10 and the blank after comma denotes selecting all columns. Using “aq\$Ozone > 20”, we can set logical indexing to return all rows where Ozone is greater than 20. The rows where Ozone = NA are not printed in the output but we see their indexes as blank. Subset produces a cleaner output where it displays only non-NA rows > 20.

```
{r q16-18} aq <- airquality[1:10, ] aq[aq$Ozone > 20, ] subset(aq,
Ozone > 20)
```

19. Create a TooWindy variable inside aq, which is a logical variable that is TRUE if Wind is greater than 10, and FALSE otherwise.

Using the “<-” operator, I have created a new variable TooWindy inside aq based on the logical condition of Wind > 10.

```
{r q19} aq$TooWindy <- aq$Wind > 10 aq$TooWindy
```

20. Use the length() function to determine the number of observations in the airquality dataframe.

Used length() to print the number of rows or observations in airquality dataframe.

```
{r q20} length(airquality$Ozone)
```

21. Calculate the mean and standard deviation of one of the variables in airquality.

Used “mean()” and “sd()” methods to calculate mean and standard deviation respectively of Ozone variable in airquality. Since the dataset contains NA values, we use “na.rm = TRUE” to exclude all NA values and calculate mean and standard deviation.

```
{r q21} mean(airquality$Ozone, na.rm = TRUE) sd(airquality$Ozone,
na.rm = TRUE)
```

22. Make a table of the Temp values.

Used “table()” method to create table of Temp values.

```
{r q22} table(airquality$Temp)
```

23. Make a histogram of the Ozone column. Is it a normal distribution? Why or why not?

Used “hist()” method to make a histogram of Ozone. The histogram is not a normal distribution since it is skewed on the left side and does not have the bell curve shape expected for normal distribution.

```
{r q23} hist(airquality$Ozone)
```

## Functions:

24. Make a simple function to calculate x + 6.

Used “function()” to create a function called add\_six that returns  $x + 6$  where  $x$  is the input to the function

```
{r q24} add_six <- function(x) { return(x + 6) }
```

25. Use that function add 6 to the Temp column in airquality.

Used previously defined add\_six function to Temp and get the result vector. This result vector is then assigned back to Temp column using “<-” operator

```
{r q25} airquality$Temp <- add_six(airquality$Temp)
```

## Packages:

26. Install the ggplot2 package.

27. Install the car package.

28. Install the ez package. (no output necessary for these three)

29. Load the car library.

Used “install.packages()” to install all required packages and “library()” to load required library

```
{r q29} install.packages("ggplot2") install.packages("car")  
install.packages("ez") library(car)
```

## Files

30. Import the csv file provided on Canvas.

Used “read.csv()” to load the required csv file from local directory and store it in csv\_data variable

```
{r q30} csv_data <- read.csv("lab_R_learning.csv")
```